

Agent 当前开发现状

本文盘点当前仓库中已经开发的 Agent 能力、模块边界、前后端入口和已暴露的问题。它描述的是现状，不是理想设计。

1. 总体定位

当前 Agent 已经不是简单聊天框，而是一套面向 `/analysis` 工作台的 GIS 分析编排系统。

它已经覆盖：

- 用户问题识别
- 当前地图范围和分析快照整理
- LLM 规划
- 工具选择
- GIS 工具执行
- 证据审计
- 审计后重规划
- 结构化综合回答
- 会话持久化
- 流式进度展示
- 附件检索
- 报告总结生成
- 区域画像和选址场景包

当前主链路更接近一个轻量 Agent harness，而不是单次问答接口。

2. 后端主链路

Agent 主流程位于 `modules/agent/runtime.py`。

当前阶段包括：

阶段	作用	主要模块
<code>gating</code>	判断用户问题是否可执行，是否缺范围或关键意图	<code>gate.py</code>
<code>context_ready</code>	整理当前 <code>analysis snapshot</code> 和已有 <code>artifacts</code>	<code>context_builder.py</code>
<code>planning</code>	生成分析目标、证据关注点和是否需要工具	<code>planner.py</code> 、 <code>providers/llm_provider.py</code>
<code>executing</code>	调用注册工具并收集结果	<code>executor.py</code> 、 <code>tool_service.py</code>
<code>auditing</code>	检查证据是否足够回答问题	<code>auditor.py</code>
<code>replanning</code>	审计不通过时补充工具步骤	<code>runtime.py</code> 、 <code>planner.py</code>

阶段	作用	主要模块
synthesizing	生成结构化结论、证据、建议和卡片	synthesizer.py
answered	返回最终回答并持久化会话	session_service.py

主链路已经具备完整闭环，但目前自由度较高，容易在简单任务上产生过长执行链。

3. API 入口

Agent 路由集中在 `router/domains/agent.py`。

当前已暴露接口包括：

接口	用途
POST <code>/api/v1/analysis/agent/turn</code>	非流式 Agent turn
POST <code>/api/v1/analysis/agent/turn/stream</code>	流式 Agent turn
POST <code>/api/v1/analysis/agent/react/run</code>	创建 ReAct run
GET <code>/api/v1/analysis/agent/react/stream</code>	ReAct run 流式事件
POST <code>/api/v1/analysis/agent/react/cancel/{run_id}</code>	取消 ReAct run
GET <code>/api/v1/analysis/agent/sessions</code>	获取 Agent 会话列表
GET <code>/api/v1/analysis/agent/sessions/{session_id}</code>	获取会话详情
PUT <code>/api/v1/analysis/agent/sessions/{session_id}</code>	保存会话快照
PATCH <code>/api/v1/analysis/agent/sessions/{session_id}</code>	更新会话元数据
DELETE <code>/api/v1/analysis/agent/sessions/{session_id}</code>	删除会话
POST <code>/api/v1/analysis/agent/context-ask</code>	针对报告块或图表的上下文解释
POST <code>/api/v1/analysis/agent/site-selection</code>	生成选址分析包
GET <code>/api/v1/analysis/agent/tools</code>	列出 Agent 工具
POST <code>/api/v1/analysis/agent/attachments</code>	上传附件
GET <code>/api/v1/analysis/agent/attachments</code>	获取附件列表
DELETE <code>/api/v1/analysis/agent/attachments/{attachment_id}</code>	删除附件
GET <code>/api/v1/analysis/agent/prompts</code>	列出 prompt 配置
GET <code>/api/v1/analysis/agent/prompts/{prompt_key}</code>	读取单个 prompt
PUT <code>/api/v1/analysis/agent/prompts/{prompt_key}</code>	更新 prompt
POST <code>/api/v1/analysis/agent/summary/readiness</code>	检查报告总结数据就绪状态
POST <code>/api/v1/analysis/agent/summary/generate</code>	流式生成总结报告包

接口	用途
POST /api/v1/analysis/agent/iteration/nightlight/interpret	夜光迭代解释
POST /api/v1/analysis/agent/iteration/poi/interpret	POI 迭代解释
POST /api/v1/analysis/agent/iteration/poi/build	构建 POI 迭代 payload

接口数量已经较多，后续需要区分“核心 Agent 主链路”和“附属分析服务”。

3.1 Agent 输入体系

当前 Agent 的输入已经不只是用户在聊天框输入的一段文字，而是由多种上下文共同组成。

现有输入来源包括：

输入类型	来源	说明
用户文本	Agent composer / chat messages	用户直接提出的问题、追问、分析目标
当前地图范围	analysis_snapshot.scope	当前等时圈、绘制范围、polygon、isochrone feature
当前分析快照	AnalysisSnapshot	POI、H3、人口、夜光、路网、前端分析面板、过滤条件等
已有运行产物	memory.artifacts	工具执行后产生的结构化中间结果
历史分析上下文	modules/retrieval/service.py	当前会话内的 analysis chunks 和 report chunks
用户上传附件	modules/retrieval/attachments.py	PDF、图片、Office、表格、文本等外部材料
附件检索结果	search_uploaded_attachment_context / read_uploaded_attachment_context	Agent 可按问题检索并读取上传文件中的证据片段

3.1.1 附件上传

附件上传入口位于：

- POST /api/v1/analysis/agent/attachments
- GET /api/v1/analysis/agent/attachments
- DELETE /api/v1/analysis/agent/attachments/{attachment_id}

前端对应逻辑主要在：

- frontend/src/features/agent/runtime.js
- frontend/src/features/agent/normalizers.js

- `frontend/src/features/agent/session-store.js`

上传后附件会保存到：

```
runtime/agent_uploads/{conversation_id}/{attachment_id}/
```

每个附件会有：

```
source/      原始文件
rag/         RAG-Anything 工作目录
parsed/      解析输出目录
metadata.json
chunks.json
```

附件状态包括：

```
uploaded
processing
ready
failed
```

前端会轮询附件状态，只有 `ready` 状态的附件会作为 `attachment_ids` 传入 Agent turn。

3.1.2 支持的附件类型

允许的扩展名配置在 `core/config.py`：

```
.pdf
.jpg / .jpeg / .png / .bmp / .tiff / .tif / .gif / .webp
.doc / .docx
.ppt / .pptx
.xls / .xlsx
.txt
.md
```

默认最大附件大小为 30MB。

3.1.3 开源解析能力

当前项目已经接入：

```
raganything[all]==1.3.1
```

配置项包括：

```
RAGANYTHING_PARSER=mineru|docling|paddleocr
RAGANYTHING_PARSE_METHOD=auto|ocr|txt
RAGANYTHING_EMBEDDING_MODEL=text-embedding-3-large
RAGANYTHING_EMBEDDING_DIM=3072
```

也就是说，现在已经具备接收多模态/多格式材料的基础能力：

- PDF 文档
- 图片
- 图纸截图
- 表格
- Word 文档
- PPT
- Markdown / TXT
- 带图表、公式、表格的材料

解析流程在 `modules/retrieval/attachments.py` 中。

核心流程是：

```
用户上传附件
-> 保存原始文件
-> 创建 AttachmentRecord
-> 后台调用 RAG-Anything
-> 解析文本、图片、表格、公式等内容
-> 生成可检索上下文
-> 切成 AttachmentChunk
-> 写入 chunks.json
-> 附件状态变为 ready
```

3.1.4 Agent 如何使用附件

附件不会直接整份塞进 LLM prompt。

Agent 通过工具检索：

工具	作用
<code>search_uploaded_attachment_context</code>	根据用户问题搜索上传附件中的相关片段
<code>read_uploaded_attachment_context</code>	读取搜索命中的具体 chunk

工具定义在：

- `modules/agent/tool_definitions/retrieval.py`

工具实现位于：

- `modules/agent/tool_adapters/retrieval_tools.py`

Prompt 中已经明确要求：

- 用户提到附件、文件、图片、图纸、表格、报告时，优先检索附件。
- 附件证据必须标注文件名。
- 附件内容不能伪装成地图分析计算结果。

3.1.5 当前输入体系的问题

目前输入能力已经比较强，但还需要收敛：

1. 附件输入和 GIS snapshot 输入还没有形成统一的“证据类型”模型。
2. 附件 chunk 是外部材料证据，不能和 POI、人口、夜光等计算结果混为一谈。
3. 附件检索目前依赖 Agent 自己决定何时调用，常见场景可以做显式入口，例如“基于这份规划文件分析当前范围”。
4. RAG-Anything 解析结果被二次 query 后切 chunk，后续可考虑保存更细的页码、表格、图片定位。
5. 多附件、多轮追问时，需要更清楚地展示“本次回答引用了哪些文件证据”。

这部分是 Agent 成为“城市规划研究员”的重要输入层：它不仅能读地图数据，也能读用户上传的规划文本、图纸截图、表格和汇报材料。

4. LLM Provider 和多角色 Prompt

LLM 调用集中在 `modules/agent/providers/`。

当前已经实现：

- OpenAI-compatible / DeepSeek 风格 chat completions 调用
- JSON role 调用
- 流式 reasoning delta
- tool calling 消息解析
- tool call 合并与执行
- title 生成
- Gatekeeper、Planner、Tool Selector、Auditor、Synthesizer、Tool Loop prompt

主要文件：

- `providers/client.py`
- `providers/llm_provider.py`
- `providers/prompts.py`
- `providers/tool_loop.py`
- `providers/tool_call_execution.py`
- `providers/chat_parser.py`

当前 LLM 角色包括：

角色	作用
Gatekeeper	判断是否可执行、是否需要澄清或阻断

角色	作用
Planner	生成分析目标、问题类型和证据关注点
Tool Selector	根据 Planner 意图选择工具步骤
Auditor	判断证据是否足够，必要时要求重规划
Synthesizer	生成最终结构化输出
Tool Loop	ReAct/tool calling 风格的工具调度

5. 工具注册和工具执行

工具系统已经比较完整，核心文件包括：

- tools.py
- tool_service.py
- executor.py
- tool_definitions/
- tool_adapters/

工具按定义和适配器分离。

5.1 工具定义

工具定义位于 modules/agent/tool_definitions/：

文件	作用
common.py	RegisteredTool、ToolSpec 和注册 helper
foundation.py	基础工具，如读取范围、读取结果、基础数据计算
capability.py	能力工具，如数据就绪、下一步分析、空间结构分析
scenario.py	场景工具，如区域画像、选址分析
analysis_business.py	商业分析相关工具
retrieval.py	分析上下文、报告上下文、附件检索工具

5.2 工具适配器

工具适配器位于 modules/agent/tool_adapters/。

当前已开发：

文件	能力
scope_tools.py	读取和归一化当前范围
result_tools.py	读取当前已有分析结果
poi_tools.py	POI 获取

文件	能力
h3_tools.py	H3 网格和指标
population_tools.py	人口概览
nightlight_tools.py	夜光概览
road_tools.py	路网句法
spatial_cell_tools.py	空间同格对齐
capability_tools.py	数据就绪、下一步分析、POI/空间结构/标签/候选评分
analysis_tools.py	读取和生成结构化分析证据
business_tools.py	商业选址建议
scenario_tools.py	区域画像包、选址包
retrieval_tools.py	analysis/report/attachment 检索

工具系统已经能覆盖 POI、H3、人口、夜光、路网、空间同格、区域画像、选址、检索等核心 GIS 分析能力。

6. 城市分析抽取器

modules/agent/analysis_extractors.py 是当前 Agent 最重要的领域能力底座之一。

它已经包含：

- POI 结构分析
- H3 结构分析
- 路网模式分析
- 人口画像分析
- 夜光模式分析
- POI 业态混合分析
- 商业热点识别
- 目标业态供给缺口
- 区域特征标签推断
- 候选点评分

这些能力使 Agent 不只是复述数据，而是能把已有数据加工成城市规划语义。

7. 场景包

当前已经有场景化分析包，主要在 tool_adapters/scenario_tools.py。

已实现：

- run_area_character_pack
- run_site_selection_pack
- run_placeholder_scene_pack

其中：

- `run_area_character_pack` 用于区域画像、商业调性、功能标签、活动信号、客群特征等判断。
- `run_site_selection_pack` 用于目标业态选址、候选区、供给缺口和评分。

后续更适合继续扩展为稳定 recipe，而不是让通用 Agent 对所有问题自由规划。

8. 审计和边界控制

当前已经有两类控制：

8.1 证据审计

`auditor.py` 会检查：

- 是否有 POI 证据
- 是否有 H3 密度证据
- 是否有路网证据
- 是否有人口证据
- 是否有夜光证据
- 是否有 POI 结构分析
- 是否有 H3 结构分析
- 是否有目标候选证据
- 是否需要空间同格对齐

审计不通过时，主链路会进入 `replanning`。

8.2 工具治理

`governance.py` 会检查工具风险和治理模式。

Agent 支持：

- `auto`
- `guarded`
- `readonly`

高风险或高成本工具可以要求用户确认。

9. 综合输出

`synthesizer.py` 负责把证据和工具结果转为最终输出。

当前输出结构包括：

- `decision`
- `support`
- `counterpoints`
- `actions`
- `boundary`
- `cards`
- `next_suggestions`
- `review_contract`

- `panel_payloads`

它已经试图把 Agent 输出从普通文本升级为结构化城市规划判断。

当前风险是：综合阶段输入偏大，复杂任务容易触发 JSON 输出失败，需要进一步压缩输入和增加后端 fallback。

10. Context Ask

`context_ask_service.py` 已经实现针对某个 target 的解释能力。

它的定位是：

- 只解释当前点击对象
- 不重新规划
- 不调用工具
- 使用 target、evidence、artifact_refs 和 snapshot summary
- 证据不足时给出缺口说明

如果后续将“追问”统一定义为“继续分析”，这个服务可以下沉为 follow-up context builder 的内部能力，而不是单独作为用户侧追问主入口。

11. Summary Pack

`summary_service.py` 已经实现报告总结相关能力。

包括：

- 数据就绪检查
- 面板 payload 生成
- 报告结构生成
- 分 section LLM 生成
- follow-up questions
- tourism cross analysis
- 流式 summary 事件

该模块已经很大，属于 Agent 体系下的报告生成子系统。

12. Iteration 分析

当前已经有迭代解释相关服务：

- `iteration_change_service.py`
- `poi_iteration_build_service.py`

已支持：

- 夜光迭代解释
- POI 迭代解释
- POI 多年份趋势 payload 构建
- 增长区识别
- 分类变化摘要

- 空间趋势证据增强

这部分更偏“专题分析服务”，可以被 Agent 调用或由前端直接触发。

13. LangGraph / ReAct 尝试

当前已有 LangGraph ReAct loop：

- `providers/langgraph_react.py`
- `react_orchestrator.py`

它支持：

- 创建 run
- 流式事件
- tool calling
- max steps
- max errors
- cancel
- trace/action/observation/final 事件

这说明项目已经开始尝试接入开源 Agent runtime，但目前主链路仍主要是自研 orchestrator。

14. 会话持久化

`session_service.py` 和 `store/agent_session_repo.py` 支撑 Agent 会话。

当前支持：

- 会话列表
- 会话详情
- 创建 / 更新会话
- 更新元数据
- 删除会话
- turn 结果持久化
- 会话标题生成
- 会话状态、输出、诊断、上下文、计划和附件保存

前端也已经配套做了 session store 和 UI 状态同步。

15. 前端 Agent 工作区

前端 Agent 相关代码主要在：

- `frontend/src/features/agent/runtime.js`
- `frontend/src/features/agent/session-store.js`
- `frontend/src/features/agent/sessions-ui.js`
- `frontend/src/features/agent/normalizers.js`
- `frontend/src/features/agent/derived.js`
- `frontend/src/pages/analysis/components/agent/`

当前已支持：

- Agent 输入
- 流式执行
- thinking timeline
- reasoning panel
- trace 展示
- plan 展示
- 会话列表
- 会话切换
- 会话重命名 / 删除 / pin
- clarification card
- risk confirmation
- 附件状态
- Agent panel payload 预加载
- pending task confirmation
- summary task log tracking

前端已经承载了大量 Agent 运行态逻辑，后续可以考虑把 prompt 拼接和业务判断进一步收回后端。

16. 已有测试

Agent 前端已有测试：

- `frontend/tests/agent-sessions.test.js`
- `frontend/tests/agent-normalizers.test.js`
- `frontend/tests/agent-derived.test.js`

这些主要覆盖 session、normalizer、derived UI 状态。

后端 Agent 主链路、工具 recipe、审计规则和 synthesis fallback 仍需要更明确的测试覆盖。

17. 当前主要问题

从现状看，Agent 已经“做得很多”，但出现了几个明显结构问题。

17.1 自由编排过重

简单任务也会经过 Gatekeeper、Planner、Tool Selector、Executor、Auditor、Replanner、Synthesizer，多轮 LLM 参与后容易耗时过长。

典型问题：

- “总结商业特征”这类固定任务可能跑成 10 多步。
- Planner 判断证据充分，Auditor 又要求补证据，随后又重新判断充分。
- 用户看到大量内部过程，但得不到稳定结论。

17.2 常见任务缺少固定 recipe

目前已经有区域画像和选址场景包，但主链路仍倾向让 LLM 自由决定工具链。

更适合固定为 recipe 的任务包括：

- 区域商业特征总结
- 业态缺口分析
- 咖啡 / 餐饮 / 便利店选址
- 夜间活力诊断
- 路网可达性诊断
- TOD 初筛
- 15 分钟生活圈缺口
- 更新优先级判断

17.3 Synthesizer 上下文过大

综合阶段目前会接收较多 payload，包括 snapshot digest、context digest、synthesis payload、tool results、metrics、evidence matrix 等。

复杂任务下容易导致：

- LLM JSON 输出损坏
- 响应时间过长
- 用户最后只看到 LLM 调用失败

17.4 UI 暴露了过多内部过程

前端展示了大量 planning、audit、replan、tool trace 文本。

对开发调试有价值，但对演示用户来说噪声较大。

用户更需要看到：

- 正在检查哪些证据
- 当前结论是什么
- 哪些证据支持
- 哪些地方不确定
- 下一步能做什么

17.5 证据异常没有足够硬约束

部分异常信号应该阻止模型得出强结论。

例如：

- 同格对齐后 `active_poi_cell_count=0`
- H3 网格为空
- 夜光有效像素过少
- 路网为空或节点过少
- 前端 analysis key 存在但 payload 为空

这些应该作为 rule audit 的硬边界，而不是交给模型自由解释。

18. 建议的后续收敛方向

下一步不建议继续横向增加 Agent 功能，而应优先收敛成几个稳定的城市规划场景工作流。

建议优先级：

1. 把“区域商业特征总结”做成固定 recipe。
2. 把“追问”统一为 `continue_analysis`，并携带 anchor 和当前 analysis snapshot。
3. 限制 Synthesizer 输入体积，只传 evidence digest，不传完整 tool results。
4. 增加结构化 fallback：LLM 综合失败时仍返回后端构造的 cards。
5. 隐藏大部分内部 trace，把 UI 主视图改成用户可理解的分析进度。
6. 对关键证据异常增加硬规则审计。
7. 明确哪些接口属于核心 Agent，哪些属于专题服务或历史实验。

19. 简短结论

当前 Agent 已经具备完整技术链路：

```
+++ 用户问题
+++ 分析快照
+++ LLM 规划
+++ 工具注册
+++ GIS 工具执行
+++ 证据审计
+++ 结构化输出
+++ 前端流式展示
+++ 会话持久化
```

但产品层面还需要从“通用自由 Agent”收敛为“稳定城市规划场景 Agent”。

换句话说，下一阶段的重点不是继续让 Agent 更自由，而是让它在几个高频规划任务上更短、更准、更稳、更像专业分析师。